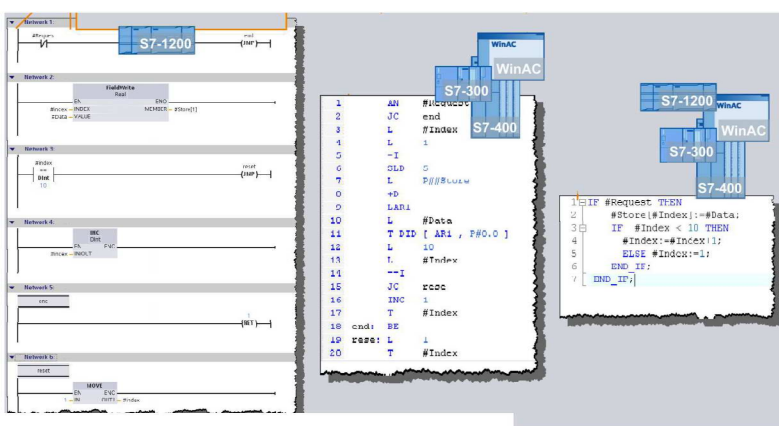


Structured Text a high level language



Operators in ST

Symbol	Operation
(expression)	Parenthesization
identifier(argument list)	Function evaluation
Examples:	
LN(A), MAX(X,Y), etc.	
**	Exponentiation
-	Negation
NOT	Complement

Operators in ST

*	Multiply
/	Divide
MOD	Modulo
+	Add
-	Subtract
<, >, <=, >=	Comparison
=	Equality
<>	Inequality

C-language

$a = b$
 $a \neq b$

Operators in ST

&	Boolean AND
AND	Boolean AND
XOR	Boolean Exclusive OR
OR	Boolean OR

Operators and Expressions

()	parenthesized expression
function()	function
**	exponentiation
-	negation
NOT	Boolean complement
+ - *	math operators
MOD	modulus operation
< > <= >=	comparison operators
=	equal
<>	not equal
AND, &	Boolean AND
XOR	Boolean XOR
OR	Boolean OR

light	door	out
T	T	
T	F	
F	T	
F	F	

```

Y := X+1.0;

y := a AND b;

v := (v1 + v2 + v3)/3

output := (light = open) OR (door = shut);

```

Variable Declarations

Local variable:

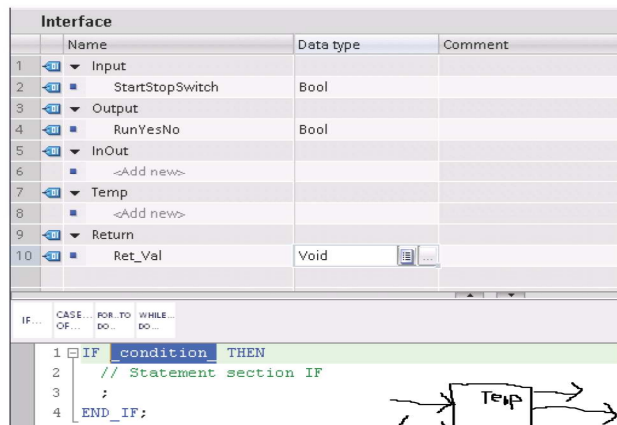
```

VAR
  Ij,k : INT;
  v : REAL;
END_VAR

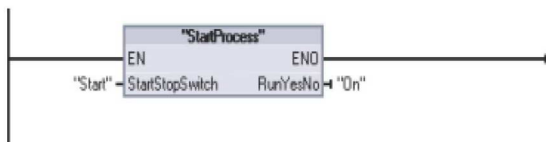
```

Use VAR, VAR_INPUT, VAR_OUTPUT, VAR_IN_OUT, VAR_GLOBAL, VAR_EXTERNAL for different variable types.

ST language (SCL) in Simatic PLC



- **Input** - input parameters, corresponding to the inputs to the block, placed on the left side of the block symbol in LAD language,
- **Output** - output parameters, corresponding to the outputs from the block, placed on the right side of the block symbol in the LAD language,
- **InOut** - parameters that are an input to a block, and at the same time in a block can be changed and output to the outside,
- **Static** - (Available only in FB function blocks) - static local variables that are visible inside a block but retain their value between different block calls,
- **Temp** - local variables, visible inside the block and removed at the end of the block call, do not retain their value.



assignment

A := B;
INT to INT
or

Analog_Channel_Configuration to Analog_Channel_Configuration

CV := CV+1;
C := SIN(X);

Variable declarations

Name	Data_type
01. Input	
02. inp	INT;
03. Return	
04. res	INT;

Function code

```

05. (*****
The function returns the square of the input
value, or when the square exceeds the permissible
range of an integer, the maximum permissible
integer number.
09. *****)
10.
11. IF #inp <= 181 THEN
12. #res := #inp * #inp; //Calculation of the function value
13. ELSE
14. #res := 32767; // When an overflow would occur...
15. END_IF;

```

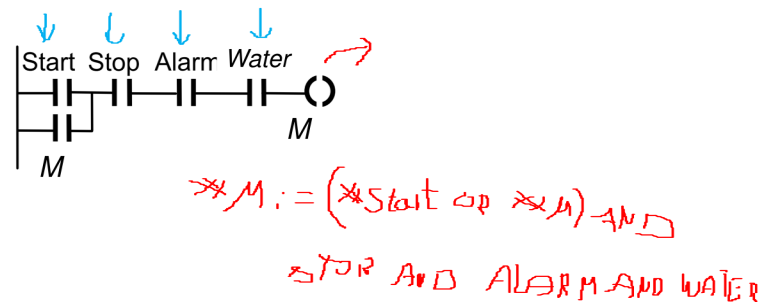
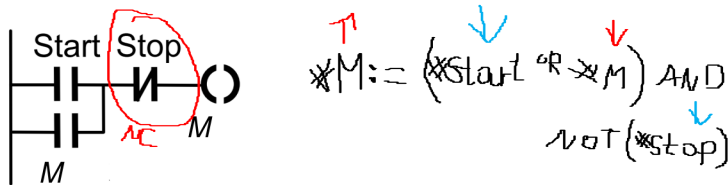
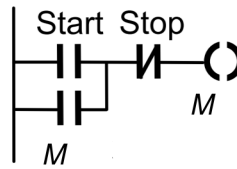
:=	Assignment instructions. A variable or array elements may appear on the left side, expression. on the right side #a:=#b + #c - into a variable enter the sum of variables b and c
()	Round brackets are used to determine the sequence of actions.
+, -, *, /	Arithmetic operators: sum, difference, multiplication and division
**	Exponentiation #a :=#b ** 4 - a:=b*b*b*b
MOD	Operator reszty z dzielenia. Wartość 17 MOD 4 jest równa 1
<, <=, >, >=, =, <>	Comparison operators, respectively: smaller, no bigger, bigger, no smaller, equal, different
NOT	Logical negation operator
AND, &	Logical product operator (selectable, one of the symbols)
OR	Logical sum operator
XOR	The operator of the exclusion sum A XOR B is true when only one of the values of A, B is true
#<nazwa>	Reference to the local variable <name> #inp
"<nazwa>"	Reference to the global variable <name> "System_State"
<nazwa>	Reference to absolute address I0.0
[]	Square brackets are used to indicate the array element. #Tab[#Index] element of the local array #Tab indicated by the variable #Index

Examples of assignment instructions:

- "C" := #A+#B;
the sum of the local variables #A and #B is entered in the global variable "C".
- "Data_block_1".State := #A;
"The value of the local variable #A is entered in the Status field of the global data block "Data_block_1".
- #C := SQRT (SQR (#A) + SQR (#B));
a value equal to the square root of the sum of the squares of the local variables #A and #B is entered into the local variable #C; the function SQR determines the square of the number, the function SQRT - the square root of the number

```
#Water:= 0 ;
#"Water(high)" :=1;
//If the tag has an operator in its name, it is completed with
quotation marks
"Air" :=True;
"MyTag1" := "MyTag2";
"MyTag1" := "MyTag2" * "MyTag3";
"MyTag" := "MyFC"();

#MyStruct.MyStructElement := "MyTag";
#MyArray[2] := "MyTag"
"MyTag" := #MyArray[1,4];
#MyString[2] := #MyOtherString[5]
```



Controlled repetition via iteration or selection

➤ IF .. THEN .. ELSE

➤ CASE

➤ FOR

➤ WHILE ...

➤ REPEAT UNTIL

IF .. THEN .. ELSE

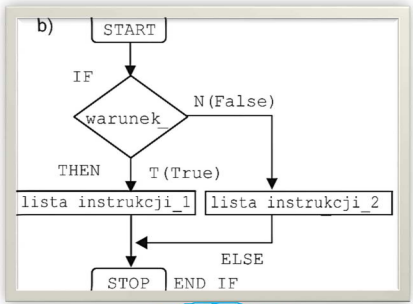
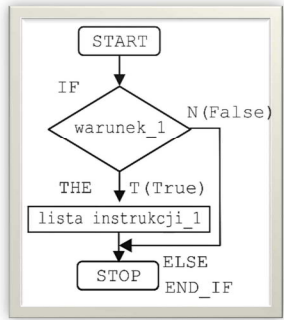
```
IF condition= true
THEN action1
ELSE action2(condition not true)
END_IF
```

```
IF raining THEN
    Stay_In;
ELSE
    Go_Out;
END_IF
```

C- language
if(raining) Stay_In;
else {Go_Out};

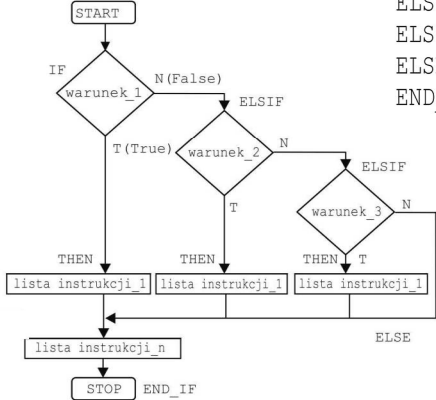
Favorites	
IF... THEN... ELSE...	IF... THEN... ELSE... (T...F)
FOR... TO... DO...	FOR... TO... DO...
WHILE... DO...	WHILE... DO...
REPEAT... UNTIL...	REPEAT... UNTIL...
Basic Instructions	
Bit logic operations	IF... THEN... instrukcja wykonania warunku
Timer operations	IF... THEN... ELSE... instrukcja wykonania kilku warunków
Counter operations	IF THEN ELSE IF... instrukcja wykonania kilku warunków wielokrotnie
Math functions	CASE... OF... instrukcja wyboru
Move operations	FOR... TO... DO... instrukcja pętli
Conversion operations	FOR... TO... BY... DO... instrukcja pętli z wartością skoku
Program control operations	WHILE... DO... instrukcja pętli, jeżeli warunek spełniony
scd IF... THEN...	REPEAT... UNTIL... instrukcja pętli, jeżeli warunek niespełniony
scd IF... THEN... ELSE...	CONTINUE natychmiastowe przejście do kolejnej iteracji
scd IF... THEN... ELSEIF...	EXIT opuszczenie natychmiastowe pętli
scd CASE... OF...	GOTO skok
scd FOR... TO... DO...	RETURN opuszczenie bloku
scd FOR... TO... BY... DO...	Wprowadzenie komentarza do bloku
scd WHILE... DO...	RE_TRIGR restart czasu monitorowania cyklu
scd REPEAT... UNTIL...	STP opuszczenie programu
scd CONTINUE	GET_ERROR wywołanie błędu lokalnego
scd EXIT	INIT_RD wywołanie ID błędu lokalnego
scd GOTO...	WAIT opóźnienie wykonania programu w ms (tylko w S7-1500)
scd RETURN	RUNTIME pomiar czasu pracy programu (tylko w S7-1500)
scd Insert block comment	
Runtime control	
RE_TRIGR	
STP	
GET_ERROR	
GET_ERROR_ID	
INIT_RD	
WAIT	
RUNTIME	
Word logic operations	
Shift and rotate	
Additional instructions	

```
IF <warunek1> THEN <lista instrukcji1>;
[ELSIF <warunek2> THEN <lista instrukcji2>];
[ELSE <lista instrukcji>];
END IF;
```

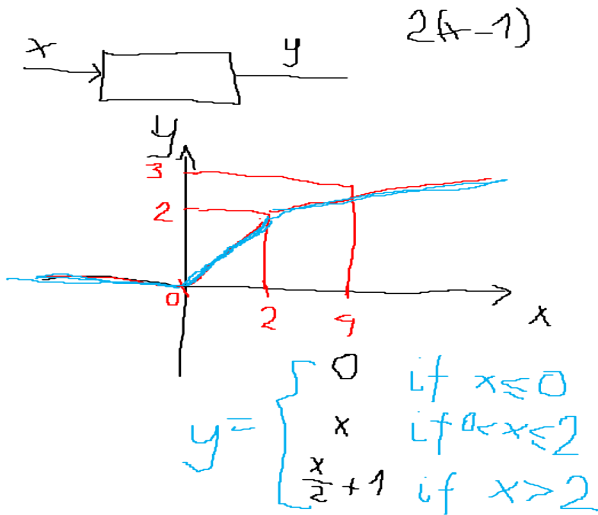


```
IF A>0 AND B>1 THEN
    x:=10; y:=20;
ELSE
    x:=0; y:=0;
END IF;
```

```
IF <warunek1> THEN <lista instrukcji1>;
[ELSIF <warunek2> THEN <lista instrukcji2>];
[ELSE <lista instrukcji>];
END IF;
```



```
If C>0 THEN x:=10;
ELSIF B=1 THEN x:=1;
ELSIF B=2 THEN x:=2;
ELSE x:=0;
END IF;
```



IF .. THEN .. ELSE

```
IF ABS(A)<EPS THEN
    IF B=0.0 THEN
        IF C=0.0 THEN
            NROOTS:=3; (* identity equation *)
        ELSE
            NROOTS:=-1; (* contradictory equation *)
        END IF
    ELSE
        NROOTS := 1; (* the root of the linear equation *)
        X1 := - C/B;
    END IF
ELSE
    D := B*B - 4.0*A*C;
    IF D < 0.0 THEN
        NROOTS := 0; (* lack of real roots *)
    ELSIF D = 0.0 THEN
        NROOTS := 1; (* double root *)
        X1 := - B/(2.0*A);
    ELSE
        NROOTS := 2; (* two roots *)
        X1 := (- B + SQRT(D))/(2.0*A);
        X2 := (- B - SQRT(D))/(2.0*A);
    END IF;
END IF;
```

CASE : repetitive IF

```
CASE selector OF
    selector_value : action
    selector_value : action
    selector_value : action
ELSE
    action
END_CASE;
```

```
CASE A OF
    1:x:=1;
    2,5:x:=2;
    6..10:x:=3;
    11,12,15:x:=4;
ELSE y:=0;
END_CASE
```

CASE : repetitive IF

```
TW := BCD_TO_INT(THUMBWHEEL);
```

```
TW_ERROR := 0;
```

```
CASE TW OF
```

```
  1,5:  DISPLAY := OVEN_TEMP;
  2:    DISPLAY := MOTOR_SPEED;
  3:    DISPLAY := GROSS - TARE;
  4,6..10: DISPLAY := STATUS(TW - 4);
  ELSE  DISPLAY := 0;
        TW_ERROR := 1;
```

```
END_CASE;
```

```
QW100 := INT_TO_BCD(DISPLAY);
```



Condition Statements

```
IF a > 100 THEN
```

```
  redlight := on;
```

```
ELSEIF a > 50 THEN
```

```
  yellowlight := on;
```

```
ELSE
```

```
  greenlight := on;
```

```
END_IF;
```

```
CASE dial_setting OF
```

```
  1 :   x := 10;
```

```
  2 :   x := 15;
```

```
  3 :   x := 18;
```

```
  4,5:  x := 20; (* 4 or 5 *)
```

```
ELSE
```

```
  x := 30;
```

```
END_CASE
```

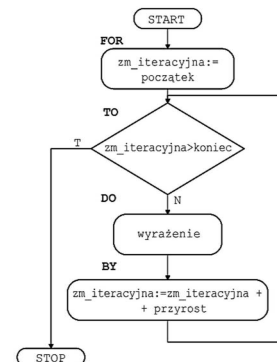
FOR

```
SUM := 0 ;
```

```
FOR begin_value TO end_value DO
```

```
  ...
```

```
END_FOR ;
```



```
max:=0;
min:=100;
FOR i:= 0 TO 100 BY 1 DO
  IF data[i]>max THEN
    max:=data [i];
  END_IF;
  IF data[i]<min THEN
    min:=data [i];
  END_IF;
END_FOR;
```

FOR

```
SUM := 0 ;
```

```
FOR I := 1 TO 3 DO
```

```
  FOR J := 1 TO 2 DO
```

```
    SUM := SUM + J ;
```

```
  END_FOR ;
```

```
  SUM := SUM + I ;
```

```
END_FOR ;
```

SUM=?

CONTINUE, EXIT

```
FOR (WHILE, REPEAT) <wyrażenie>;
```

```
  ...
```

```
  IF <warunek> THEN CONTINUE; // Jeżeli warunek nie jest spełniony, wykonywany jest program za instrukcją
```

```
END_IF;
```

```
  ...
```

```
END_FOR (WHILE, REPEAT);
```

```
FOR i:=1 TO 15 BY 2 DO
```

```
  IF (i < 5) THEN CONTINUE;
```

```
END_IF;
```

```
"DB10".Test[i] := 1;
```

```
END_FOR;
```

CONTINUE, EXIT

```
FOR (WHILE, REPEAT) <wyrażenie>;  
...  
IF <warunek> THEN EXIT; // Jeżeli warunek jest spełniony, wykonywany jest program  
                           poza pętlą  
END_IF;  
...  
END_FOR (WHILE, REPEAT);
```

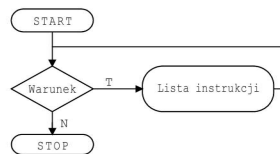
```
SUM := 0 ;  
FOR I := 1 TO 3 DO  
    FOR J := 1 TO 2 DO  
        IF FLAG THEN EXIT ; END_IF  
        SUM := SUM + J ;  
    END_FOR ;  
    SUM := SUM + I ;  
END_FOR ;
```

```
WHILE <warunek> DO  
<lista instrukcji>;  
END_WHILE;
```

WHILE

```
n:=1;  
WHILE x<3000 DO  
    x:=x*2;  
    data [n]:= x;  
    n:=n+1;  
END_WHILE;
```

```
J := 1;  
WHILE J <= 100 & WORDS[J] <> 'KEY' DO  
    J := J+2 ;  
END_WHILE ;
```

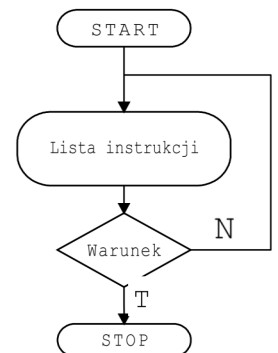


```
REPEAT  
    <lista instrukcji>  
    UNTIL <warunek>;  
END_REPEAT;
```

```
A:=0;  
TOTAL:=0;  
REPEAT  
    TOTAL:= TOTAL+A;  
    A:=A+1;  
    UNTIL A>10;  
END_REPEAT;
```

```
J := -1 ;  
REPEAT  
    J := J+2 ;  
UNTIL J = 101 OR WORDS[J] = 'KEY'  
END_REPEAT ;
```

REPEAT ... UNTIL



Iteration Statements

```
FOR I:= 0 TO 100 BY 1 DO  
    light[I] := ON;  
END_FOR
```

```
I := 0;  
WHILE I < 100 DO  
    I := I + 1;  
    light[I] := on;  
END_WHILE
```

```
I := 0;  
REPEAT  
    I := I + 1;  
    light[I] := on;  
UNTIL I > 100;  
END_REPEAT
```

Functions

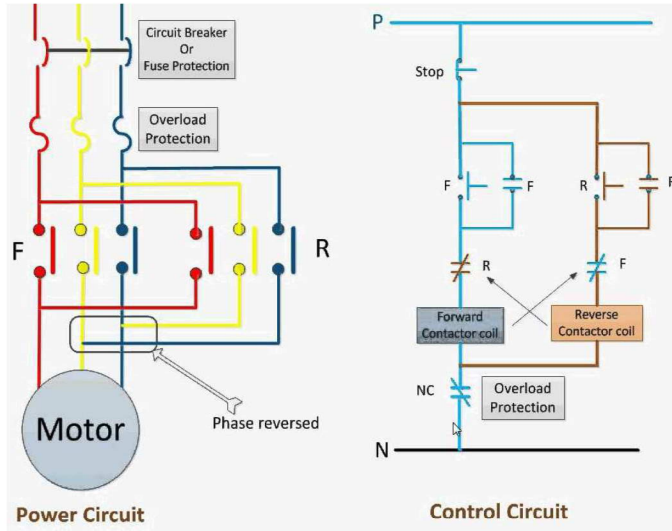
```
FUNCTION add_num    : REAL  
    VAR_INPUT  
        I,J :    REAL  
    END_VAR  
    add_num := I + J;  
END_FUNCTION
```

Call a function:

```
x:= add_num(1.2, 5.6);
```

Built-in Functions:

ABS, SQRT, LN, LOG, EXP, SIN, COS, TAN, ASIn, ACOS, ATAN, ADD, MUL, SUB, DIV, MOD, EXPT, MOVE), logic functions (AND, OR, XOR, NOT), bit string functions (SHL, SHR shift bit string left and right , ROR, ROL rotate bit string), etc.



```
VAR
    ButtonForward:BOOL;
    ButtonReverse:BOOL;
    ButtonStop:BOOL;
    MotorForward:BOOL;
    MotorReverse:BOOL;
    IndicatorForward:BOOL;
    IndicatorReverse:BOOL;

    SlopeForward : R_TRIG;
    SlopeReverse : R_TRIG;
    CoilForward : RS;
    CoilReverse : RS;
    RunOutForward : TOF;
    RunOutReverse : TOF;
```

```
END_VAR
```

```
SlopeForward(CLK := ButtonForward);
SlopeReverse(CLK := ButtonReverse);
```

```
CoilForward(SET:=SlopeForward.Q AND NOT(IndicatorReverse),
    RESET1:=NOT(ButtonStop));
MotorForward:=CoilForward.Q1;
CoilReverse(SET:=SlopeReverse.Q AND NOT(IndicatorForward),
    RESET1:=NOT(ButtonStop));
MotorReverse:=CoilReverse.Q1;
```

```
RunOutForward(IN :=MotorForward, PT := T#10S);
IndicatorForward :=RunOutForward.Q;
```

```
RunOutReverse(IN :=MotorReverse, PT := T#10S);
IndicatorReverse :=RunOutReverse.Q;
```

```
01. IF #inp1<0.0 THEN
02.   #out1:=0; // if #inp1<0
03. ELSIF #inp1<5.0 THEN
04.   #out1:=1; // if 0<=#inp1<5.0
05. ELSIF #inp1<10.0 THEN
06.   #out1:=2; // if 5.0<=#inp1<10.0
07. ELSE
08.   #out1:=3; // if #inp1>=10.0
09. END_IF;
```

```
01. CASE #inp1 OF
02.   0:   #out1:=-1.5; // if #inp1=0
03.   1:   #out1:=-0.5; // if #inp1=1
04.   2..4: #out1:=1.5; //if #inp1=2 or #inp1=3 or #inp1=4
05.   ELSE: #out1:=2.5; // other case
06. END_CASE;
```

```
01. #inter:=1
02. WHILE (# inter *2)<=#inp1 DO
03.   #inter := #inter *2;
04. END_WHILE;
05. #out1:=#inter;
```

```
01. #inter :=1
02. REPEAT
03.   #inter := #inter *2;
04. UNTIL #inter >=#inp1;
05. #out1:=#inter /2;
```

```
01. #inter :=1;
02. FOR #i := 1 TO #inp1 DO
03.   #inter := #inter *2;
04. END_FOR;
05. #out1:=#inter;
```

```
01. Input
02.   start      BOOL;
03.   ref_time   TIME
03. Output
04.   out1       BOOL;
05.   out2       BOOL;
06. Temp
07.   restart    BOOL;
08.   time       TIME
09.
10. "IEC_Timer_0_DB_2".TON(IN:=(#start & NOT(#restart)),
11.   PT:=# ref time,
12.   Q=>#restart,
13.   ET=># time);
14.
15. IF # time <(# ref_time /2) THEN
16.   #out1:=0;
17. ELSE
18.   #out1:=1;
19. END_IF;
20. IF # time <(#1s) THEN
21.   #out2:=#start;
22. ELSE
23.   #out2:=0;
24. END_IF;
```

```
01. "IEC_Counter_0_DB_2".CTUD(CU:=_bool_in_,
02.                               CD:=_bool_in_,
03.                               R:=_bool_in_,
04.                               LD:=_bool_in_,
05.                               PV:=_in_,
06.                               QU=>_bool_out_,
07.                               QD=>_bool_out_,
08.                               CV=>_out_);
```